UNIQUE Key in MySQL

What is UNIQUE Key?

A **UNIQUE key** is a constraint in MySQL that ensures:

No two rows can have the same value in that column (or column combination).

It prevents duplicate data.

Simple Meaning

If a column should contain only distinct values, use UNIQUE.

Examples:

- Email
- Username
- Passport number
- Employee code

Example Table

```
CREATE TABLE users (
  id INT PRIMARY KEY,
  email VARCHAR(100) UNIQUE
);
```

Now allowed:

id	email
1	a@gmail.com
2	b@gmail.com

Not allowed:

id	email
3	a@gmail.com

Duplicate email rejected.

What Error Comes?

```
INSERT INTO users VALUES (3,'a@gmail.com');
```

Error:

Duplicate entry 'a@gmail.com' for key

Why We Need UNIQUE Key

To protect business data.

Without it:

- two users same login email
- duplicate employee codes
- repeated account numbers
- messy customer records

Real-Time Examples

---

1. User Registration

email UNIQUE

One email = one account.

---

2. Company Employees

employee_code UNIQUE

No duplicate employee IDs.

---

3. Products

sku UNIQUE

Each product has unique stock code.

Difference: UNIQUE vs PRIMARY KEY

Feature	UNIQUE	PRIMARY KEY
Prevent duplicates	Yes	Yes
NULL allowed	Yes (MySQL allows)	No
One per table	Many possible	Only one

Feature	UNIQUE	PRIMARY KEY
Main identity	No	Yes

Important: NULL in UNIQUE

In MySQL, a UNIQUE column can often contain multiple NULLs because NULL means unknown.

Example:

```
CREATE TABLE users (
 email VARCHAR(100) UNIQUE
);
```

Allowed:

email

NULL

NULL

Because NULL ≠ NULL logically.

(Behavior can vary by DB system)

Deep Internal Working

When you create UNIQUE key, MySQL usually creates a **unique index**.

That index helps:

- fast lookups
- duplicate checking during insert/update

Example Internally

If table has emails:

a@gmail.com

b@gmail.com

c@gmail.com

When inserting:

b@gmail.com

MySQL checks index quickly and rejects duplicate.

Add UNIQUE to Existing Table

```
ALTER TABLE users
ADD CONSTRAINT uq_email UNIQUE(email);
```

Important Warning

If old duplicates already exist:

a@gmail.com

a@gmail.com

Then adding UNIQUE fails.

Need cleanup first.

Check duplicates:

```
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Composite UNIQUE Key

Combination of columns must be unique.

Example:

```
CREATE TABLE attendance (
 student_id INT,
 class_date DATE,
 UNIQUE(student_id, class_date)
);
```

Meaning:

Same student cannot have two attendance rows for same date.

Update Also Checks UNIQUE

Not only insert.

If update causes duplicate:

```
UPDATE users
SET email='a@gmail.com'
WHERE id=2;
```

Rejected if another row already has it.

UNIQUE vs DISTINCT

Many beginners confuse these.

Term	Meaning
UNIQUE	Table rule / constraint
DISTINCT	Query result removes duplicates

Example

```
SELECT DISTINCT city FROM users;
```

This is query output only.

Not table rule.

Real Data Engineering Importance

UNIQUE helps with:

- deduplication
- clean dimensions
- trusted joins
- CDC merge keys
- preventing duplicate customer records

Best Columns for UNIQUE

Use fields naturally unique:

- email
- username
- employee_code
- SKU

- tax_id

Avoid unstable fields unless needed.

Final One-Line Answer

A UNIQUE key in MySQL prevents duplicate values in a column or column combination while allowing each row to stay distinct.

Interview Answer

A UNIQUE constraint enforces uniqueness of values and is commonly used for emails, usernames, and business identifiers.

Honest Advice

In real systems, UNIQUE constraints save enormous cleanup effort later. Prevent duplicates early rather than fixing them in analytics later.

Difference Between PRIMARY KEY vs UNIQUE KEY

Both are used to prevent duplicates, but they are **not the same**.

Quick Definition

Primary Key

Main column used to uniquely identify each row.

Unique Key

Column that must have unique values, but is not the main identity.

Main Differences

Feature	PRIMARY KEY	UNIQUE KEY
Prevent duplicates	Yes	Yes
NULL allowed	No	Yes (MySQL allows NULLs)
Count per table	Only 1	Many allowed

Feature	PRIMARY KEY	UNIQUE KEY
Main row identifier	Yes	No
Used for foreign key target commonly	Yes	Yes (sometimes)
Automatically indexed	Yes	Yes

Example Table

```
CREATE TABLE users (
  user_id INT PRIMARY KEY,
  email VARCHAR(100) UNIQUE,
  username VARCHAR(50) UNIQUE
);
```

Here:

- user_id = main identity
- email = must be unique
- username = must be unique

Why Only One Primary Key?

Because a table should have **one main identity**.

Like a person can have:

- Aadhaar
- Passport
- Employee ID

But in one system, one chosen main identifier.

Why Many UNIQUE Keys?

Because many fields may need no duplicates.

Example users table:

- email unique
- username unique
- mobile unique

NULL Difference (Important)

PRIMARY KEY

NULL not allowed

UNIQUE KEY

In MySQL:

NULL usually allowed, multiple NULLs possible

Because NULL means unknown.

Real Example

user_id	email
1	a@gmail.com
2	b@gmail.com
3	NULL
4	NULL

Allowed if email has UNIQUE in MySQL.

But user_id cannot be NULL.

Business Example

Employees Table

emp_id PRIMARY KEY

email UNIQUE

pan_no UNIQUE

Meaning:

- emp_id = official row identity
- email and PAN also must not repeat

Deep Internal Meaning

Primary Key Usually Used For:

- joins

- foreign keys
- row lookup
- clustering (some DBs)
- core table identity

Unique Key Usually Used For:

- alternate candidate keys
- business uniqueness rules

Candidate Key Concept

Possible unique identifiers:

- user_id
- email
- username

Choose one as **Primary Key**. Others become **UNIQUE keys**.

Example Insert Errors

Duplicate Primary Key

INSERT INTO users VALUES (1,'x@gmail.com');

Fails if id=1 exists.

Duplicate Unique Email

INSERT INTO users VALUES (5,'a@gmail.com');

Fails if email exists.

Interview-Level Summary

Primary Key

Entity integrity.

Unique Key

Business uniqueness.

Best Practice

Use:

- Surrogate ID as primary key (user_id)
- Natural unique fields as unique keys (email)

Very common in real companies.

Memory Trick

Primary = Main Unique Identity

Unique = Additional No-Duplicate Rule

Final One-Line Answer

Primary key is the main non-null unique identifier of a row, while unique key is any other column that must also avoid duplicates.

Honest Advice

In real design, use a stable integer/UUID primary key and put UNIQUE on business fields like email. That scales better than using email as primary key.

DEFAULT in MySQL

What is DEFAULT?

DEFAULT in MySQL is used to assign an **automatic value** to a column when no value is provided during INSERT.

If user does not give a value, MySQL fills it automatically.

Simple Meaning

Instead of manually entering same value every time, database sets it for you.

Example

```
CREATE TABLE users (
  id INT PRIMARY KEY,
  status VARCHAR(20) DEFAULT 'ACTIVE'
);
```

Now run:

INSERT INTO users(id) VALUES (1);

Stored row becomes:

id	status
1	ACTIVE

Because no status was provided.

Why We Need DEFAULT

To avoid:

- repeated typing
- missing common values
- inconsistent manual entries
- nulls where default value makes sense

Real-Time Examples

---

1. User Accounts

status DEFAULT 'ACTIVE'

New users active by default.

---

2. Orders

order_status DEFAULT 'PENDING'

Every new order starts pending.

---

3. Employees

country DEFAULT 'India'

Useful if most users are from one country.

---

4. Timestamps

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Auto stores creation time.

Syntax

While Creating Table

```
CREATE TABLE students (
  id INT,
  city VARCHAR(50) DEFAULT 'Bangalore'
);
```

Add Later

```
ALTER TABLE students
ALTER city SET DEFAULT 'Hyderabad';
```

(MySQL syntax can differ by version)

Often:

```
ALTER TABLE students
MODIFY city VARCHAR(50) DEFAULT 'Hyderabad';
```

How It Works Internally

When you run:

```
INSERT INTO users(id) VALUES(5);
```

MySQL checks columns.

For omitted status:

- if default exists → use it
- else if nullable → NULL
- else if NOT NULL no default → may error (strict mode)

Example with Explicit Value

```
INSERT INTO users VALUES (2,'BLOCKED');
```

Stored:

id	status
2	BLOCKED

Because provided value overrides default.

Important Rule

DEFAULT is used only when column value is omitted (or sometimes DEFAULT keyword used explicitly).

Example: Explicit NULL

INSERT INTO users VALUES (3,NULL);

If column allows NULL:

Result = NULL, not ACTIVE.

Because you explicitly gave NULL.

DEFAULT + NOT NULL

Very common combo.

status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'

Meaning:

- cannot be NULL
- if omitted, becomes ACTIVE

Excellent design.

DEFAULT with Numbers

salary DECIMAL(10,2) DEFAULT 0

DEFAULT with Dates

created_on DATE DEFAULT (CURRENT_DATE)

(Depends on version/type support)

DEFAULT CURRENT_TIMESTAMP

Very common.

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

When row inserted:

MySQL stores current date/time automatically.

Remove Default

ALTER TABLE users

ALTER status DROP DEFAULT;

or modify column depending version.

See Default Values

DESC users;

or

SHOW CREATE TABLE users;

Common Mistakes

---

1. Thinking DEFAULT updates old rows

No. It affects future inserts only.

---

2. Thinking DEFAULT applies when NULL explicitly inserted

No. Explicit NULL stays NULL (if allowed).

---

3. Bad Defaults

Example:

email DEFAULT 'test@gmail.com'

Terrible design — creates duplicates/confusion.

Real Data Engineering Importance

Defaults reduce dirty data in source systems:

- missing statuses
- missing timestamps
- blank lifecycle stages

Cleaner source = easier pipelines.

Best Practice Examples

status DEFAULT 'NEW'

is_active DEFAULT 1

created_at DEFAULT CURRENT_TIMESTAMP

quantity DEFAULT 0

country DEFAULT 'India'

DEFAULT vs NULL

Situation	Result
Column omitted, has default	Default used
Column omitted, no default, nullable	NULL
Column omitted, NOT NULL strict	Error
Explicit NULL inserted	NULL (if allowed)

Interview Answer

DEFAULT sets an automatic value for a column when no value is supplied during insert.

Final One-Line Answer

DEFAULT in MySQL automatically fills a column with a predefined value when the insert statement does not provide one.

Honest Advice

Use DEFAULT for stable common values (status, timestamps, flags), not for business data users should intentionally choose.

AUTO_INCREMENT in MySQL

What is AUTO_INCREMENT?

AUTO_INCREMENT is a MySQL feature that automatically generates the next numeric value for a column when a new row is inserted.

You don’t manually type the ID. MySQL creates it.

Usually used with:

- Primary Key
- ID columns
- Order numbers
- Employee IDs

Simple Meaning

Every new row gets a new number:

1, 2, 3, 4, 5...

Example

```
CREATE TABLE students (
  student_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50)
);
```

Now insert:

```
INSERT INTO students(name) VALUES ('Indra');
INSERT INTO students(name) VALUES ('Ravi');
INSERT INTO students(name) VALUES ('Ajay');
```

Table becomes:

student_id	name
1	Indra
2	Ravi
3	Ajay

Why We Need AUTO_INCREMENT

Without it, you must manually write IDs:

```
INSERT INTO students VALUES (1,'Indra');
INSERT INTO students VALUES (2,'Ravi');
```

Problems:

- duplicate IDs
- forgetting next number
- race conditions with many users

AUTO_INCREMENT solves this.

Real-Time Examples

---

1. Users Table

user_id INT AUTO_INCREMENT PRIMARY KEY

Every user gets unique ID.

---

2. Orders Table

order_id INT AUTO_INCREMENT PRIMARY KEY

Each order gets new number.

---

3. Employees Table

emp_id INT AUTO_INCREMENT PRIMARY KEY

How It Works Internally

MySQL stores the current next value.

When insert happens:

INSERT INTO students(name) VALUES ('Kiran');

MySQL:

1. checks current next id
2. assigns it
3. stores row
4. increases next counter

Example Sequence

Next value = 1

Insert row → gets 1

Next value = 2

Insert row → gets 2

Next value = 3

Important Rule

Usually used on indexed key column:

PRIMARY KEY

If You Omit ID Column

INSERT INTO students(name) VALUES ('Deepa');

MySQL auto-fills ID.

If You Give Manual ID

INSERT INTO students(student_id,name)

VALUES (10,'John');

MySQL uses 10 (if valid).

Then future next values may continue after highest value.

Example

Rows:

1

2

3

10

Next auto id may become:

11

Deleting Rows Does Not Reuse Normally

If row 2 deleted:

Rows:

1

3

4

Next insert becomes:

5

Not 2.

Why Gaps Happen

IDs may skip because of:

- deleted rows
- rolled back inserts
- failed transactions
- manual inserts

This is normal.

Reset Starting Value

ALTER TABLE students AUTO_INCREMENT = 100;

Next inserted row starts from 100 (if allowed).

Get Last Inserted ID

Useful in apps.

SELECT LAST_INSERT_ID();

Gets ID generated in current session.

Big Important Note

AUTO_INCREMENT gives **unique IDs**, not guaranteed gapless sequence.

Do not use for invoice/legal sequence where no gaps allowed.

INT Size Matters

If huge table:

BIGINT AUTO_INCREMENT

Better than INT for billions of rows.

AUTO_INCREMENT vs UUID

Feature	AUTO_INCREMENT	UUID
Human readable	Yes	No
Small/faster index	Yes	Larger
Sequential	Yes	No
Distributed uniqueness	Harder	Strong

Many companies use both depending need.

Real Data Engineering Importance

Surrogate keys in warehouses often use generated IDs.

Source OLTP systems commonly use AUTO_INCREMENT IDs.

Common Mistakes

1. Expecting no gaps

Wrong.

2. Using business meaning in ID

Wrong. ID should just identify row.

3. Using small INT for huge scale

May overflow later.

Interview Answer

AUTO_INCREMENT automatically generates sequential numeric values for a column, commonly used for primary keys.

Final One-Line Answer

AUTO_INCREMENT in MySQL automatically assigns the next numeric ID to new rows, usually for primary key columns.

Example Best Practice

CREATE TABLE users (

```
  user_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(100) UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Honest Advice

Use AUTO_INCREMENT for simple single-database systems. For large distributed systems, also learn UUIDs and Snowflake-style IDs.